OS LAB — ASSIGNMENT-WISE PRACTICAL CODE

Compact version based on your uploaded Mock Test Question Bank

Assignments 1–5 included | Assignments 7 and 10 excluded as requested earlier

Assignment 1 — Linux Environment and File Management

Q1. Create directory/file and list directory

Aim: Create OS_Lab, enter it, create sample.txt and list contents.

```
#!/bin/bash
mkdir -p OS_Lab       # Create directory if needed
cd OS_Lab             # Enter directory
touch sample.txt      # Create empty file
ls                   # List directory contents
```

Steps: Save as a1_q1.sh; chmod +x a1_q1.sh; ./a1_q1.sh

Expected output:

```
sample.txt
```

Q2. Create file and display content

Aim: Write a line into student.txt and print it.

```
#!/bin/bash
touch student.txt                          # Create file
echo "Operating System Lab" > student.txt # Write text
cat student.txt                            # Display file
```

Steps: chmod +x a1_q2.sh; ./a1_q2.sh

Expected output:

```
Operating System Lab
```

Q3. Copy, move, rename, delete file

Aim: Perform file operations in sequence.

```
#!/bin/bash
echo "Sample" > student.txt  # Make source file
cp student.txt copy.txt      # Copy
mv copy.txt newfile.txt      # Move/rename
mv newfile.txt renamed.txt   # Rename
rm renamed.txt              # Delete
echo "File operations completed successfully."
```

Steps: chmod +x a1_q3.sh; ./a1_q3.sh

Expected output:

```
File operations completed successfully.
```

Assignment 2 — Shell Scripting and Automation

Q6. Positive or negative

Aim: Use if-else to test the sign of input.

```
#!/bin/bash
echo "Enter a number:"
read n
if [ "$n" -ge 0 ]; then
    echo "Positive Number"
else
    echo "Negative Number"
fi
```

Steps: chmod +x a2_q6.sh; ./a2_q6.sh

Expected output:

```
Enter a number: 25\nPositive Number
```

Q7. Even or odd

Aim: Check remainder when divided by 2.

```
#!/bin/bash
echo "Enter a number:"
read n
if [ $((n % 2)) -eq 0 ]; then
    echo "Even Number"
else
    echo "Odd Number"
fi
```

Steps: chmod +x a2_q7.sh; ./a2_q7.sh

Expected output:

```
Enter a number: 10\nEven Number
```

Q8. Arithmetic menu using case

Aim: Choose addition, subtraction, multiplication, or division.

```
#!/bin/bash
read -p "Enter first number: " a
read -p "Enter second number: " b
echo "1.Add 2.Subtract 3.Multiply 4.Divide"
```

```
read -p "Enter your choice: " ch
case "$ch" in
  1) echo "Result = $((a+b))" ;;
  2) echo "Result = $((a-b))" ;;
  3) echo "Result = $((a*b))" ;;
  4) if [ "$b" -ne 0 ]; then echo "Result = $((a/b))"; else echo "Cannot divide by zero"; fi ;;
  *) echo "Invalid Choice" ;;
esac
```

Steps: chmod +x a2_q8.sh; ./a2_q8.sh

Expected output:

```
10, 5, choice 1 → Result = 15
```

Assignment 3 — Process Creation and Scheduling

Q13. fork() and process IDs

Aim: Create child and display parent/child IDs.

```
#include <stdio.h>
#include <unistd.h>
int main(void) {
    pid_t pid=fork();             // Create child
    if(pid<0) printf("Process creation failed\n");
    else if(pid==0) {             // Child branch
        printf("Child Process\n");
        printf("Child PID = %d\n",getpid());
    } else {                      // Parent branch
        printf("Parent Process\n");
        printf("Parent PID = %d\n",getpid());
    }
    return 0;
}
```

Steps: gcc a3_q13.c -o a3_q13 && ./a3_q13

Expected output:

```
Parent Process / Parent PID ... / Child Process / Child PID ...
```

Q15. FCFS CPU scheduling

Aim: Compute waiting time and turnaround time in arrival order.

```
#include <stdio.h>
int main(void) {
    int n,i,bt[20],wt[20],tat[20];
    float avgwt=0,avgtat=0;
    printf("Enter number of processes: "); scanf("%d",&n);
    for(i=0;i<n;i++) {
        printf("Enter burst time of P%d: ",i+1);
        scanf("%d",&bt[i]);
    }
    wt[0]=0;                       // First process waits zero
    for(i=1;i<n;i++) wt[i]=wt[i-1]+bt[i-1];
    printf("Process\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++) {
        tat[i]=wt[i]+bt[i];
        avgwt+=wt[i]; avgtat+=tat[i];
        printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
    }
    printf("Average WT=%.2f\nAverage TAT=%.2f\n",avgwt/n,avgtat/n);
    return 0;
}
```

Steps: gcc a3_q15.c -o a3_q15 && ./a3_q15

Expected output:

```
Burst times 5,3,2 → P1 5 0 5; P2 3 5 8; P3 2 8 10
```

Q16. SJF CPU scheduling

Aim: Sort processes by burst time and calculate waiting/turnaround.

```
#include <stdio.h>
int main(void) {
    int n,i,j,t,bt[20],p[20],wt[20],tat[20];
    printf("Enter number of processes: "); scanf("%d",&n);
    for(i=0;i<n;i++){ p[i]=i+1; printf("Burst P%d: ",p[i]); scanf("%d",&bt[i]); }
    // Sort burst times and process IDs together.
    for(i=0;i<n-1;i++) for(j=i+1;j<n;j++) if(bt[i]>bt[j]) {
        t=bt[i];bt[i]=bt[j];bt[j]=t; t=p[i];p[i]=p[j];p[j]=t;
    }
    wt[0]=0;
    for(i=1;i<n;i++) wt[i]=wt[i-1]+bt[i-1];
    printf("Process\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++){ tat[i]=wt[i]+bt[i]; printf("P%d\t%d\t%d\t%d\n",p[i],bt[i],wt[i],tat[i]);}
    return 0;
}
```

Steps: gcc a3_q16.c -o a3_q16 && ./a3_q16

Expected output:

```
Burst times 6,2,4 → P2 2 0 2; P3 4 2 6; P1 6 6 12
```

Assignment 4 — Multithreading

Q18. Create POSIX thread

Aim: Create a thread and wait for it to finish.

```
#include <stdio.h>
#include <pthread.h>
void *display(void *arg) {
    printf("Thread is running\n");
    return NULL;
}
int main(void) {
    pthread_t thread;
    pthread_create(&thread,NULL,display,NULL); // Start thread
    pthread_join(thread,NULL);                // Wait for thread
    printf("Main thread completed\n");
    return 0;
}
```

Steps: gcc a4_q18.c -o a4_q18 -pthread && ./a4_q18

Expected output:

```
Thread is running\nMain thread completed
```

Q19. Multiple threads and thread IDs

Aim: Create three threads and print their IDs.

```
#include <stdio.h>
#include <pthread.h>
void *task(void *arg) {
    printf("Thread ID = %lu\n",(unsigned long)pthread_self());
    return NULL;
}
int main(void) {
    pthread_t t1,t2,t3;
    pthread_create(&t1,NULL,task,NULL);
    pthread_create(&t2,NULL,task,NULL);
    pthread_create(&t3,NULL,task,NULL);
    pthread_join(t1,NULL); pthread_join(t2,NULL); pthread_join(t3,NULL);
    printf("All threads completed\n");
    return 0;
}
```

Steps: gcc a4_q19.c -o a4_q19 -pthread && ./a4_q19

Expected output:

```
Thread ID = (varies) × 3\nAll threads completed
```

Assignment 5 — IPC and Shared Resource Management

Q20. Pipe between parent and child

Aim: Send a message through a pipe.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
int main(void) {
    int fd[2]; char msg[]="Hello from Parent",buf[100]={0};
    pipe(fd);                          // Create pipe
    pid_t pid=fork();                  // Create child
    if(pid==0) {
        close(fd[1]);                   // Child reads only
        read(fd[0],buf,sizeof(buf));
        printf("Child received: %s\n",buf);
        close(fd[0]);
    } else {
        close(fd[0]);                  // Parent writes only
        write(fd[1],msg,strlen(msg)+1);
        close(fd[1]);
        wait(NULL);                     // Wait for child
    }
    return 0;
}
```

Steps: gcc a5_q20.c -o a5_q20 && ./a5_q20

Expected output:

```
Child received: Hello from Parent
```

Q21. Shared memory concept

Aim: Create shared memory, write/read text, then detach and remove it.

```
#include <stdio.h>
#include <sys/shm.h>
#include <string.h>
int main(void) {
    int id=shmget(IPC_PRIVATE,100,IPC_CREAT|0666); // Allocate segment
    if(id<0){perror("shmget");return 1;}
    char *str=(char *)shmat(id,NULL,0);            // Attach segment
    if(str==(char *)-1){perror("shmat");return 1;}
    strcpy(str,"Hello using Shared Memory");        // Write data
    printf("Data in shared memory: %s\n",str);
    shmdt(str);                                    // Detach
    shmctl(id,IPC_RMID,NULL);                       // Remove segment
    return 0;
}
```

Steps: gcc a5_q21.c -o a5_q21 && ./a5_q21

Expected output:

```
Data in shared memory: Hello using Shared Memory
```

Q22. Message queue communication

Aim: Send and receive a message using System V message queue.

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
struct message { long type; char text[100]; };
int main(void) {
    int id=msgget(IPC_PRIVATE,0666|IPC_CREAT); // Create queue
    struct message msg;
    msg.type=1;
    strcpy(msg.text,"Hello using Message Queue");
    msgsnd(id,&msg,sizeof(msg.text),0);         // Send message
    msgrcv(id,&msg,sizeof(msg.text),1,0);       // Receive message
    printf("Received: %s\n",msg.text);
    msgctl(id,IPC_RMID,NULL);                    // Delete queue
    return 0;
}
```

Steps: gcc a5_q22.c -o a5_q22 && ./a5_q22

Expected output:

```
Received: Hello using Message Queue
```

Quick compile/run reminders

• Shell: chmod +x file.sh && ./file.sh

• C: gcc file.c -o program && ./program

• Pthreads: gcc file.c -o program -pthread && ./program

• Process IDs and thread IDs vary. Practice from your ~/OS_Lab folder.